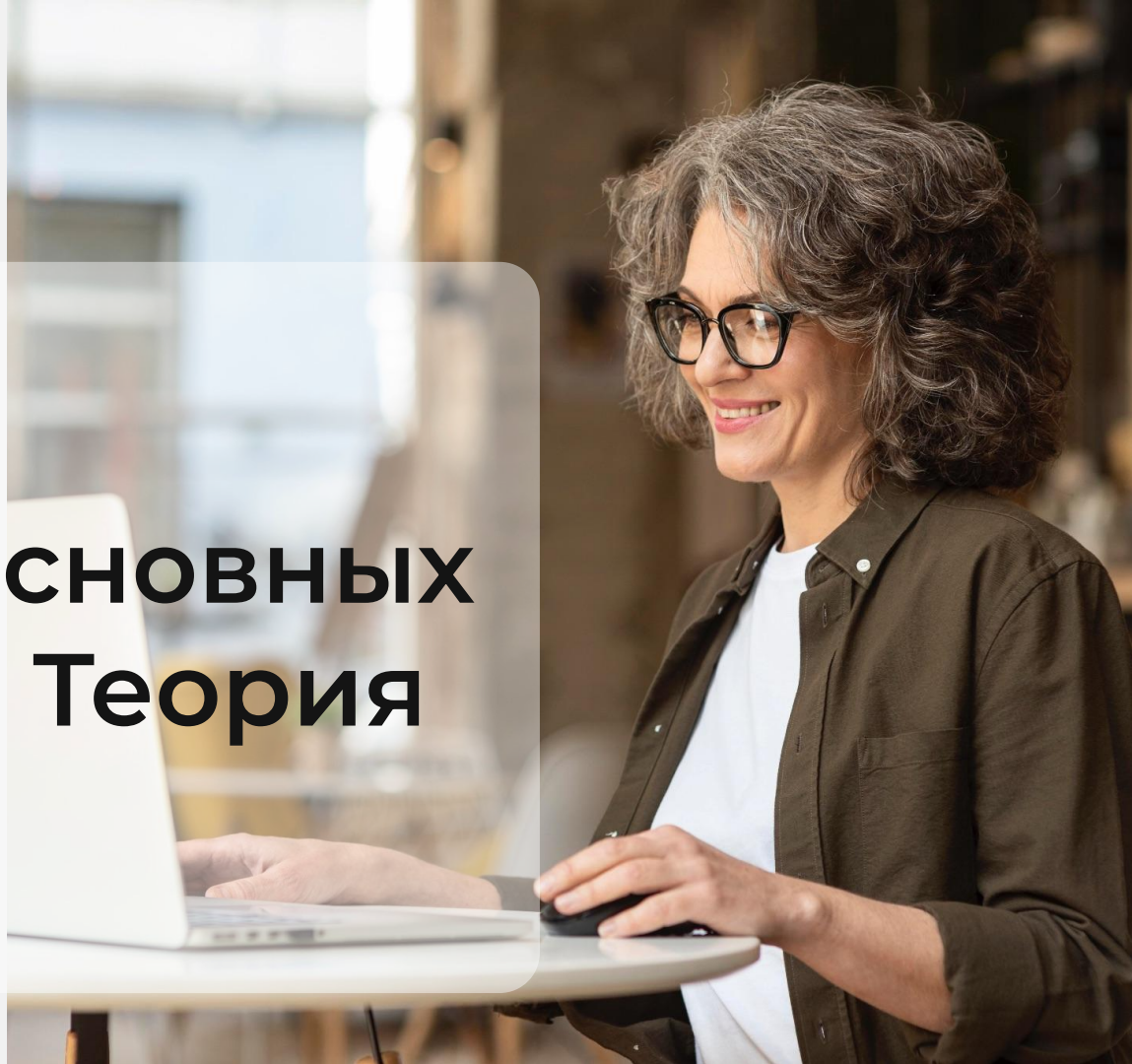


Database

Отработка основных операторов. Теория



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

-  Камера должна быть включена на протяжении всего занятия
-  В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
-  Вести себя уважительно и этично по отношению к остальным участникам занятия
-  Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
-  Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя

Повторение

-  Common Table Expression
-  Синтаксис CTE
-  Плюсы и Минусы CTE
-  Когда использовать CTE
-  Когда использовать подзапросы

План занятия

- Повторение основных операторов
- Задания на повторение



ОСНОВНОЙ БЛОК





Оператор Where



Оператор WHERE

Это оператор, который используется, чтобы указать, какие строки нужно отобрать.

Шаблон оператора WHERE

```
SELECT column1, column2
FROM table
WHERE column3 УСЛОВИЕ
```

Правила использования WHERE

Могут
применяться
любые
комбинации
констант,
литералов,
функций и имен
полей

Не является
обязательным

Все операторы
сравнения имеют
одинаковый приоритет
и выполняются в
порядке их
расположения в
выражении

Сначала выполняются
арифметические операции,
затем операции сравнения,
а в последнюю очередь —
логические операции.

Может включать
несколько
выражений,
соединенных
логическими
операторами

Логические
операторы
имеют свой
приоритет



ВОПРОСЫ





Оператор поиска по маске

Оператор поиска по маске



Оператор поиска по маске в SQL позволяет находить строки, которые соответствуют определенному шаблону. Для этой цели используется оператор LIKE в сочетании с символами подстановки (wildcards).

Поиск по маске

%

Соответствует любому количеству символов (включая ноль).

_

(Нижнее подчеркивание) соответствует одному любому символу.



ВОПРОСЫ





Оператор CASE



Оператор CASE

Это инструмент для выполнения условной логики в SQL. Он работает аналогично оператору IF-THEN-ELSE в языках программирования, позволяя возвращать разные значения на основе заданных условий.

Оператор CASE



```
CASE
  WHEN условие1 THEN результат1
  WHEN условие2 THEN результат2
  ...
  ELSE результатN
END
```

Оператор CASE

WHEN

Задаёт условие, которое проверяется. Условий может быть несколько.

THEN

Определяет результат, который возвращается, если условие выполнено.

ELSE

(Необязательно) задаёт значение, которое будет возвращено, если ни одно из условий не выполнено.

END

Завершает блок CASE.

Оператор CASE



Оператор CASE не обязательно может содержать ELSE, в таком случае все строки не попадающие под условие прописанное в WHEN ... THEN будут иметь пропуска NULL в столбце.



ВОПРОСЫ





Оператор IF



Оператор IF



Оператор IF является простым способом выполнения условной логики. Однако он поддерживается только My SQL.

Оператор IF



```
SELECT IF(condition, true_result, false_result) AS new_column_name
FROM table_name;
```


Оператор IF

condition

Условие, которое проверяется.

true_result

Возвращаемое значение, если условие истинно.

false_result

Возвращаемое значение, если условие ложно.



ВОПРОСЫ





Сортировка
результатов с
помощью ORDER BY



Оператор ORDER BY

Это оператор, который используется для сортировки результатов запроса по одному или нескольким столбцам. По умолчанию сортировка производится по возрастанию (ASC), но можно явно указать сортировку по убыванию (DESC)

Оператор ORDER BY



```
SELECT column1, column2, ...  
FROM table_name  
ORDER BY column1 [ASC|DESC], column2 [ASC|DESC], ...;
```



ВОПРОСЫ





Оператор **LIMIT**



Оператор LIMIT

Это оператор, который используется для указания максимального количества записей, которые нужно вернуть.

Оператор LIMIT



```
SELECT столбцы
FROM таблица
LIMIT количество_записей;
```



ВОПРОСЫ





Оператор **OFFSET**



Оператор OFFSET

Это оператор, который используется для пропуска определенного количества строк в результате запроса.

Оператор OFFSET



Используется в сочетании с LIMIT для реализации постраничного вывода (pagination), где LIMIT указывает количество строк на странице, а OFFSET — с какой строки начинать выборку.

Оператор OFFSET



```
SELECT column1, column2, ...
FROM table_name
ORDER BY column_name
LIMIT number_of_rows OFFSET offset_value;
```



ВОПРОСЫ





Агрегирующие функции



Агрегирующие функции

Это функции, которые позволяют производить вычисления над набором строк, возвращая одно итоговое значение.

Основные агрегирующие функции

COUNT()

Подсчитывает количество строк в наборе данных.

Можно использовать COUNT(column_name) для подсчета строк (не NULL).

SUM()

Вычисляет сумму значений в столбце.

Работает только с числовыми типами данных.

AVG()

Вычисляет среднее значение по столбцу.

Игнорирует NULL значения при вычислении среднего.

MIN()

Определяет минимальное значение в столбце.

Работает с числовыми, строковыми и датами.

MAX()

Определяет максимальное значение в столбце.

Работает с числовыми, строковыми и датами.



ВОПРОСЫ





Оператор **GROUP BY**



GROUP BY

Это оператор SQL, который используется для группировки строк, имеющих одинаковые значения в одном или нескольких столбцах.

Использование GROUP BY



SQL Server группирует строки по указанным столбцам.



Все строки с одинаковыми значениями в этих столбцах объединяются в одну группу.



Агрегирующая функция применяется к каждой группе отдельно.

- `column_name` — столбец, по которому производится группировка.
- `AGGREGATE_FUNCTION(column_name)` — агрегирующая функция, применяемая к каждой группе.

```
SELECT column_name,
AGGREGATE_FUNCTION(column_name)
FROM table_name
GROUP BY column_name;
```

Использование GROUP BY



Все столбцы в SELECT, которые не используются в агрегирующих функциях, должны быть перечислены в GROUP BY



ВОПРОСЫ





Оператор **HAVING**



Оператор HAVING

Это оператор, который применяется после оператора GROUP BY и используется для фильтрации результатов агрегатных функций на основе заданных условий.

Оператор HAVING



Позволяет отобрать только те группы, которые соответствуют определенным условиям.



GROUP BY может использоваться без HAVING, но HAVING никогда не используется без GROUP BY.

- `столбец1` — столбец, по которому происходит группировка.
- `агрегатная_функция(столбец2)` — агрегатная функция, применяемая к группам.
- `условие_для_группы` — условие, определяющее, какие группы попадут в итоговый результат.

```
SELECT столбец1, агрегатная_функция(столбец2)
FROM таблица
WHERE условия
GROUP BY столбец1
HAVING условие_для_группы;
```



ВОПРОСЫ





Операторы UNION и UNION ALL



Операторы UNION и UNION ALL

Это операторы, которые используются для горизонтального объединения результатов двух или более SELECT запросов в один результирующий набор.

- При объединении строк удаляет полные дубликаты
- Требуется очень много памяти

```
SELECT * FROM table1
UNION
SELECT * FROM table 2
```

UNION

- исключает дубликаты и возвращает уникальные строки из всех объединенных запросов.

```
SELECT * FROM table1
UNION ALL
SELECT * FROM table 2;
SELECT name, email FROM table1
UNION ALL
SELECT name, email FROM table2
```

UNION ALL

Основные особенности

UNION автоматически удаляет дубликаты, если нужно сохранить все дубликаты, используется UNION ALL.

Все запросы, объединяемые с помощью UNION, должны иметь одинаковое количество столбцов, и соответствующие столбцы должны иметь совместимые типы данных.



ВОПРОСЫ





Операторы JOIN

Операторы JOIN



- **таблица1 и таблица2** — это таблицы, которые вы хотите соединить.
- **условие_совпадения** — это логическое условие, обычно выражение `таблица1.колонка = таблица2.колонка`, которое определяет, как строки из одной таблицы соединяются со строками из другой.

```
SELECT столбцы
FROM таблица1
INNER/LEFT/RIGHT JOIN
таблица2
ON таблица1.колонка = таблица2.колонка;
```

Общий синтаксис JOINов



ВОПРОСЫ



Заключение



Вопросы наугад

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

Выберите верный вариант ответа

1. Какие операторы создают новые вычисляемые столбцы в выборке?

- a. IF
- b. ORDER BY
- c. WHERE
- d. CASE



**Вернуться к
вопросам**

Выберите верный вариант ответа

2. Какой оператор применяется чтобы вывести уникальные значения столбца?

- a. FROM
- b. IF
- c. DISTINCT
- d. WHERE



**Вернуться к
вопросам**

Выберите верный вариант ответа

3. Что такое сортировка? Возможно несколько вариантов ответов

- a. Процесс упорядочивания данных по определенному столбцу или столбцам от большего к меньшему
- b. Фильтрация
- c. Получение выборки без дубликатов
- d. Процесс упорядочивания данных по определенному столбцу или столбцам в алфавитном порядке



**Вернуться к
вопросам**

Ответьте на вопрос

4. Какое число будет выведено при выполнении запроса:

```
SELECT COUNT(DISTINCT name) FROM table
```

Данные в столбце name:
'Платон', 'Аристотель', 'Ницше', 'Кант', 'Гегель', 'Юнг', 'Платон'



Вернуться к
вопросам

Выберите верный вариант ответа

5. Какое самое главное отличие операторов IF и CASE?

- a. с помощью IF мы можем разделить данные по условию только на две группы. В CASE мы можем прописать несколько условий и, таким образом, получить новый столбец с более чем 2 категориями (группами)
- b. оператор IF позволяет нам отсортировать данные по возрастанию, а оператор CASE по убыванию
- c. Никаких отличий нет
- d. CASE применяется только для числовых столбцов, а IF для столбцов со строками



Вернуться к
вопросам

Выберите верный вариант ответа

6. Какой оператор Вы будете использовать для того, чтобы отсортировать данные по убыванию (от большего к меньшему)?

- a. ORDER BY column_name
- b. ORDER BY column_name DESC



Вернуться к
вопросам

Выберите верный вариант ответа

7. Какие уникальные значения будут в столбце grade?
Возраст сотрудников (age) — от 18 до 99 лет

```
SELECT
    age,
    CASE
        WHEN age BETWEEN 18 AND 25 THEN "junior"
        WHEN age > 25 AND age < 40 THEN "middle"
    END AS grade
FROM table;
```

- a. "junior", "middle", "senior"
- b. "junior", "middle"
- c. "junior", "middle", NULL
- d. "junior", "middle", ""



Вернуться к
вопросам

Выберите верный вариант ответа

8. Выберите правильный синтаксис (написание) оператора CASE

- a. CASE IF ... THEN ELSE
- b. CASE THEN ... WHEN ELSE ... END
- c. CASE WHEN ... THEN ELSE ... END
- d. IF CASE ... END



Вернуться к
вопросам

Ответьте на вопрос

9. Что такое алиас (alias)?



Вернуться к
вопросам

Ответьте на вопрос

10. Что выведет данный запрос?

```
SELECT
  CASE
    WHEN 1 = 1 THEN "Yes"
    ELSE "No"
  END
FROM table
```



Вернуться к
вопросам

Выполните задание

11. Установите соответствия между операторами и их применением:

- | | |
|-------------|--|
| a. SELECT | 1. Создает дополнительный столбец в соответствии с условиями |
| b. WHERE | 2. Сортирует данные |
| c. CASE | 3. Фильтрует данные |
| d. ORDER BY | 4. Формирует выборку |



Вернуться к вопросам

Выберите верный вариант ответа

12. Как выбрать все столбцы таблицы в запросе SELECT

Таблица *table* со столбцами *col1*, *col2*, *col3*

- a. SELECT columns FROM table
- b. SELECT * FROM table
- c. SELECT table FROM table
- d. SELECT col1, col2, col3 FROM table



Вернуться к
вопросам

Выберите верный вариант ответа

13. Какой запрос выведет одну строку (включающую столбцы name, salary) с максимальной заработной платой?

Таблица *table* со столбцами *name*, *salary*

- a. `SELECT salary FROM table WHERE salary = MAX(salary)`
- b. `SELECT * FROM table ORDER BY salary DESC`
- c. `SELECT MAX(salary) FROM table`
- d. `SELECT name, salary FROM table ORDER BY salary DESC LIMIT 1`



Вернуться к
вопросам

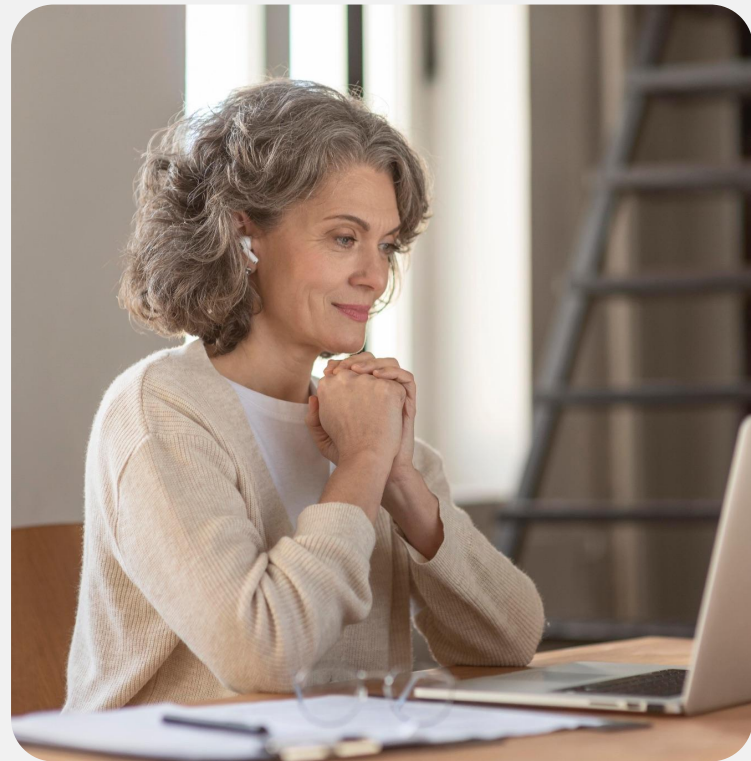
Ответьте на вопрос

14. Сколько строк выведет данный запрос?

Таблица *table* со столбцами *id* (integer), *price* (integer), *product_name* (varchar)

В таблице 10 записей

```
SELECT * FROM table
WHERE (price < 10 AND price > 10) AND price = 10
```



Вернуться к
вопросам

Ответьте на вопрос

15. Сколько строк выведет данный запрос?

Таблица *table* со столбцами *id* (integer), *price* (integer),
product_name (varchar)

В таблице 10 записей

```
SELECT * FROM table
WHERE (price < 10 OR price >= 10) OR (price = 10 AND price = 5)
```



Вернуться к
вопросам

Выполните задание

16. Установите последовательность выполнения запроса (не написания, а выполнения):

1. ORDER BY
2. WHERE
3. FROM
4. SELECT



**Вернуться к
вопросам**

Выберите верный вариант ответа

17. Как отфильтровать данные из таблицы, оставив только записи, в которых значения столбца col1 больше суммы значений столбцов col2 и col3?

Таблица *table* со столбцами *col1* (integer), *col2* (integer), *col3* (integer)

- a. `SELECT * FROM table ORDER BY col1 > col2 + col3`
- b. `SELECT *, CASE WHEN col1 > col2 + col3 THEN "1" ELSE 0 END FROM table`
- c. `SELECT * FROM table WHERE col1 > col2 + col3`
- d. `SELECT col1 > col2 + col3 FROM table`



Вернуться к
вопросам

Выберите верный вариант ответа

18. Как выбрать все записи, для которых в столбце name присутствуют пропуски

Таблица *table* со столбцами *name* (varchar), *city* (varchar)

- a. `SELECT * FROM name WHERE city = "NULL"`
- b. `SELECT * FROM table WHERE name IS NOT NULL`
- c. `SELECT * FROM table WHERE name LIKE "NULL"`
- d. `SELECT * FROM table WHERE name IS NULL`



Вернуться к
вопросам

Выберите верный вариант ответа

19. Как вывести уникальные записи из таблицы. В таблице присутствуют полные дубликаты строк
Таблица *table* со столбцами *name* (varchar), *city* (varchar)

- a. `SELECT * FROM table`
- b. `SELECT DISTINCT name FROM table`
- c. `SELECT DISTINCT city FROM table`
- d. `SELECT DISTINCT name, city FROM table`

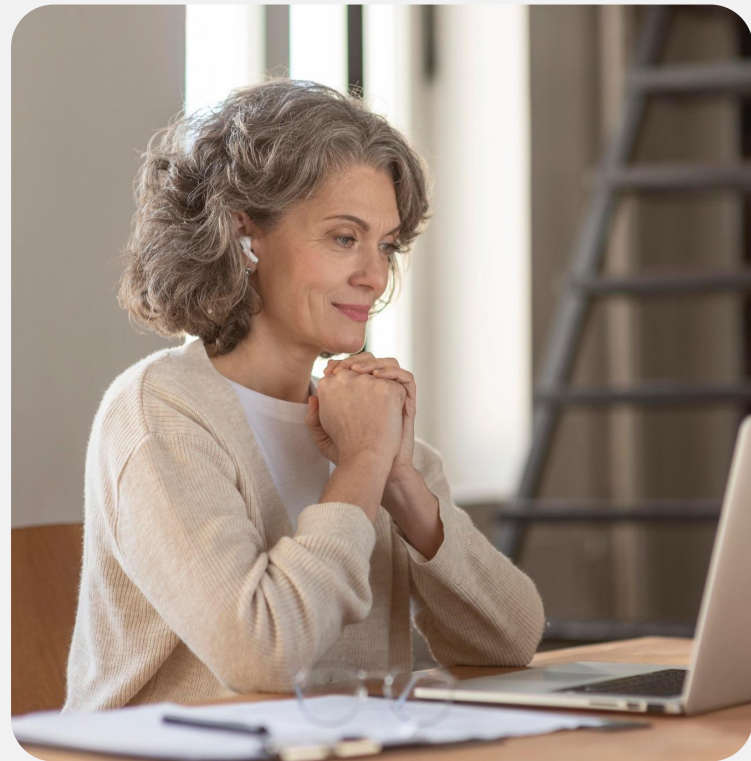


Вернуться к
вопросам

Выполните задание

20. Вывести из таблицы атрибуты `name`, `age`, `hobby`, а также дополнительный столбец со значениями: если возраст меньше 40 и хобби "fishing" — то значение "you are lazy coala", если возраст от 40 до 70 и хобби "alpinism" — значение "you are brave". В остальных случаях должны стоять пропуски

Таблица `table` со столбцами `name` (`varchar`), `age` (`integer`), `hobby` (`varchar`)



Вернуться к вопросам

Ответьте на вопрос

21. Выберите Язык Определения Данных в контексте баз данных, предназначенный для определения и управления структурой базы данных. Позволяет создавать, изменять и удалять объекты базы данных, такие как таблицы, индексы и представления.

- DML
- DDL
- SQL
- XYZ



Вернуться к
вопросам

Выполните задание

22. Установите соответствия между операторами и их действиями

- | | |
|----------------|---|
| a. FROM | 1. Вывод уникальных значений |
| b. WHERE | 2. Формирование вычисляемого столбца, в котором значение определяется выполнением условия |
| c. INSERT INTO | 3. Фильтрация данных |
| d. CREATE VIEW | 4. Сортировка данных |
| e. IF | 5. Создание представления |
| f. ORDER BY | 6. Вставка записей в таблицу |
| g. DISTINCT | 7. Выбор таблицы |



Вернуться к вопросам

Ответьте на вопрос

23. Какой запрос нужно запустить в самом начале, чтобы не прописывать имя базы данных перед именем таблицы



**Вернуться к
вопросам**

Выберите верный вариант ответа

24. Какой или какие запросы некорректны при вставке строк в таблицу

Таблица *table* со столбцами

id PRIMARY KEY NOT NULL AUTO_INCREMENT

name varchar(10) NOT NULL

address varchar(100) NOT NULL

- a. INSERT INTO table ('ddsdsd', 'qddqddqddq')
- b. INSERT INTO table (name, address) VALUES ("sdsdsdsdsdsd", "qsq")
- c. INSERT INTO table (id, name, address) VALUES(5, "sdsd", "sdqds")
- d. INSERT INTO table (name) VALUES ("sdsds")



Вернуться к
вопросам

Ответьте на вопрос

25. Можно ли одной командой полностью удалить базу данных со всеми таблицами?



**Вернуться к
вопросам**

Ответьте на вопрос

26. Как вы поняли что такое JOIN?



Вернуться к
вопросам

Выберите верный вариант ответа

**27. Что необходимо знать чтобы сдвойнить таблицы?
Возможно несколько ответов**

- a. Названия таблиц, которые мы хотим объединить
- b. Количество столбцов в каждой таблице
- c. По какому общему столбцу мы будем объединять таблицы
- d. Количество строк в каждой таблице



**Вернуться к
вопросам**

Ответьте на вопрос

28. Верно ли утверждение CROSS JOIN — это объединение таблиц, при котором мы можем не указывать по какому общему столбцу нужно объединять таблицы?



Вернуться к
вопросам

Выберите верный вариант ответа

29. Если при объединении таблиц мы напишем просто JOIN то какому из видов он будет соответствовать

- a. INNER JOIN
- b. FULL OUTER JOIN
- c. RIGHT JOIN
- d. LEFT JOIN



Вернуться к
вопросам

Ответьте на вопрос

30. Сколько строк будет в результате запроса

```
SELECT * FROM table1 INNER JOIN table2
ON table1.common_col = table2.common_col
```

если table1.common_col содержит значения (5, 12, 14, 27, 1, 88), а table2.common_col содержит значения (3, 5, 89, 2, 16, 10)



Вернуться к
вопросам

Ответьте на вопрос

31. Сколько строк будет в результате запроса

```
SELECT * FROM table1 LEFT JOIN table2
ON table1.common_col = table2.common_col
```

если table1.common_col содержит значения (5, 12, 14, 27, 1, 88), а table2.common_col содержит значения (3, 5, 89)



Вернуться к
вопросам

Ответьте на вопрос

32. Как по другому можно написать запрос

```
SELECT * FROM table1 LEFT JOIN table2
ON table1.common_col = table2.common_col
```



Вернуться к
вопросам

Выберите верный вариант ответа

33 Вид JOIN при котором в результирующую выборку попадут все записи из правой и левой таблицы. В тех записях, которые не удовлетворяют условию, в полях иной таблиц стоят null.

- a. RIGHT JOIN
- b. LEFT JOIN
- c. CROSS JOIN
- d. FULL OUTER JOIN



Вернуться к вопросам

Выберите верный вариант ответа

34. Правильный синтаксис JOIN

- a. `SELECT table1, table2 JOIN ON table1.common_col = table2.common_col`
- b. `SELECT table1 JOIN table2 ON table1.common_col = table2.common_col`
- c. `SELECT * FROM table1 JOIN table2 ON table1.common_col = table2.common_col`
- d. `FROM table1 JOIN table2 ON table1.common_col = table2.common_col SELECT *`



Вернуться к
вопросам

Выберите верный вариант ответа

35. Какой оператор позволяет нам сгруппировать данные?

- a. HAVING
- b. GROUP BY
- c. WHERE
- d. GROUP_CONCAT



Вернуться к
вопросам

Ответьте на вопрос

36. Сколько строк выведет запрос

```
SELECT order_id, MAX(order_price)
FROM Customers
GROUP BY 1
```

Таблица Customers:

- customer_first_name varchar(10) NOT NULL,
- order_price decimal(8,2) NOT NULL
- order_id integer PRIMARY KEY AUTO_INCREMENT NOT NULL

В таблице 158 строк



Вернуться к
вопросам

Выполните задание

37. Установите последовательность НАПИСАНИЯ операторов в запросе

- a. SELECT
- b. ORDER BY
- c. GROUP BY
- d. HAVING
- e. WHERE



Вернуться к
вопросам

Ответьте на вопрос

38. Корректен ли следующий запрос?

Таблица Countries:

- Country_Code integer NOT NULL,
- Country_Name varchar(20) NOT NULL
- city_id integer

```
SELECT Country_Name, COUNT(distinct city_id)
FROM Countries
GROUP BY Country_Code
```



Вернуться к
вопросам

Ответьте на вопрос

39. Зачем нам нужен оператор HAVING?



Вернуться к
вопросам

Ответьте на вопрос

40.

Можно ли использовать оператор **HAVING** без **GROUP BY**?

Можно ли использовать **GROUP BY** без **HAVING**?



Вернуться к
вопросам

Выберите верный вариант ответа

41. Какие функции относятся к агрегатным в MySQL?

- a. DATE()
- b. MIN()
- c. LENGTH()
- d. GROUP_CONCAT()
- e. GROUP BY
- f. COUNT()
- g. DISTINCT()
- h. LEFT()
- i. TRIM()
- j. SUM()



**Вернуться к
вопросам**

Ответьте на вопрос

42. Можно ли использовать агрегатную функцию без GROUP BY?



**Вернуться к
вопросам**